

Pengolahan Citra Digital

Bab 3: Pemrosesan Intensitas dan Penyaringan dalam Domain Spasial

Memahami teknik transformasi piksel, pemrosesan histogram, dan penyaringan spasial sebagai fondasi utama pengolahan citra.

Contents

1	Pendahuluan	3
2	Fondasi Matematika: Transformasi Intensitas (Bagian 3.1 & 3.2)	3
2.1	Intuisi Dasar	3
2.2	Jenis-Jenis Transformasi Dasar	3
2.2.1	Citra Negatif	3
2.2.2	Transformasi Log	4
2.2.3	Transformasi Power-Law (Gamma Correction)	4
2.3	Contoh Kode: Transformasi Intensitas dengan Python	5
3	Pemrosesan Histogram (Bagian 3.3)	6
3.1	Apa Itu Histogram Citra?	6
3.2	Histogram Equalization (HE)	6
3.3	Histogram Matching (Spesifikasi Histogram)	7
3.4	Contoh Kode: Histogram Equalization	7
4	Dasar Penyaringan Spasial (Bagian 3.4)	10
4.1	Mekanisme: Neighborhood Processing	10
4.2	Korelasi vs. Konvolusi	10
4.3	Padding: Menangani Tepi Citra	10
4.4	Contoh Kode: Konvolusi 2D Manual	10
5	Penyaringan Smoothing / Lowpass (Bagian 3.5)	11
5.1	Tujuan	11
5.2	Box Filter (Average Filter)	12
5.3	Gaussian Filter	12
5.4	Median Filter (Non-Linear)	13
5.5	Contoh Kode: Perbandingan Filter Smoothing	13
6	Penyaringan Sharpening / Highpass (Bagian 3.6)	14

6.1	Prinsip Dasar: Penajaman sebagai Diferensiasi	14
6.2	Laplacian: Turunan Orde Kedua	14
6.3	Gradient: Turunan Orde Pertama	15
6.4	Unsharp Masking	15
6.5	Contoh Kode: Sharpening dengan Laplacian dan Unsharp Masking	16
7	Ringkasan Perbandingan Filter	17
8	Mini-Latihan (Active Learning)	17

1 Pendahuluan

Pemrosesan dalam *domain spasial* adalah salah satu pendekatan paling fundamental dalam pengolahan citra digital. Berbeda dengan domain frekuensi (seperti Fourier Transform), di sini kita bekerja langsung pada nilai piksel citra.

Secara umum, operasi pada domain spasial dapat dituliskan sebagai:

$$g(x, y) = T[f(x, y)] \quad (1)$$

di mana $f(x, y)$ adalah citra masukan, $g(x, y)$ adalah citra keluaran, dan T adalah operator yang bekerja pada suatu *neighborhood* (lingkungan) di sekitar titik (x, y) .

Bab ini mencakup dua kategori besar operasi:

1. **Point Processing** – nilai piksel baru hanya bergantung pada nilai piksel itu sendiri.
2. **Neighborhood Processing** – nilai piksel baru bergantung pada piksel-piksel di sekitarnya (disebut *spatial filtering*).

2 Fondasi Matematika: Transformasi Intensitas (Bagian 3.1 & 3.2)

2.1 Intuisi Dasar

Bayangkan setiap piksel sebagai sebuah angka tunggal yang merepresentasikan kecerahan. *Transformasi intensitas* adalah sebuah fungsi yang mengambil angka lama (r) dan mengubahnya menjadi angka baru (s) berdasarkan aturan tertentu. Proses ini disebut **point processing** karena nilai baru sebuah piksel *hanya* bergantung pada nilai piksel itu sendiri, bukan tetangganya.

Rumus Umum:

$$s = T(r) \quad (2)$$

di mana r adalah intensitas sebelum diproses, s adalah intensitas sesudah diproses, dan T adalah operator transformasi.

2.2 Jenis-Jenis Transformasi Dasar

2.2.1 Citra Negatif

Membalikkan intensitas sehingga piksel terang menjadi gelap dan sebaliknya. Sangat berguna untuk menonjolkan detail putih pada latar yang gelap.

$$s = (L - 1) - r \quad (3)$$

Contoh Aplikasi: Citra mammogram sering diproses menggunakan negatif agar massa yang semula terlihat putih halus di latar terang menjadi lebih kontras. Lihat [FIGURE 3.4].

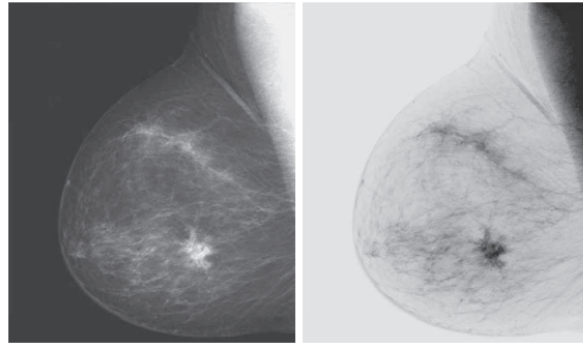


Figure 3.4: (a) Sebuah mammogram digital. (b) Citra negatif yang diperoleh menggunakan Persamaan 3.

2.2.2 Transformasi Log

Memperluas (memperjelas) nilai piksel gelap yang terkompresi. Sangat berguna ketika citra memiliki rentang nilai yang sangat ekstrem, seperti hasil spektrum Fourier.

$$s = c \cdot \log(1 + r) \quad (4)$$

di mana c adalah konstanta skala. Karena $\log(1) = 0$, nilai nol tetap dipetakan ke nol.

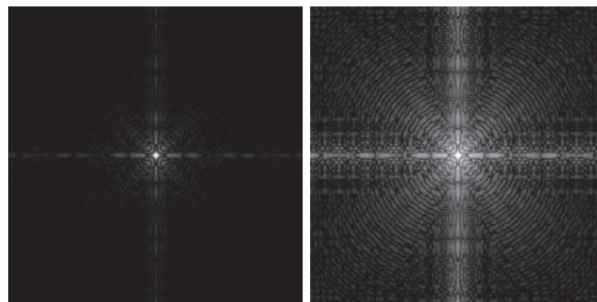


Figure 3.5: (a) Spektrum Fourier yang ditampilkan sebagai citra skala abu-abu. (b) Hasil penerapan transformasi logaritmik pada Persamaan 4 dengan $c=1$. Kedua citra diskalakan ke rentang $[0, 255]$.

Contoh Aplikasi: Menampilkan spektrum Fourier – nilai koefisien bisa berbeda jutaan kali lipat. Tanpa transformasi log, hampir semua nilai akan tampak hitam. Lihat [FIGURE 3.5].

2.2.3 Transformasi Power-Law (Gamma Correction)

Ini adalah “jantung” dari kalibrasi monitor dan pengaturan kontras citra.

$$s = c \cdot r^\gamma \quad (5)$$

- Jika $\gamma < 1$: area gelap *diperlebar* $\Rightarrow$ citra menjadi lebih terang.
- Jika $\gamma > 1$: area terang *diperlebar* $\Rightarrow$ citra menjadi lebih gelap dan kontras.
- Jika $\gamma = 1$: transformasi linier (tidak ada perubahan).

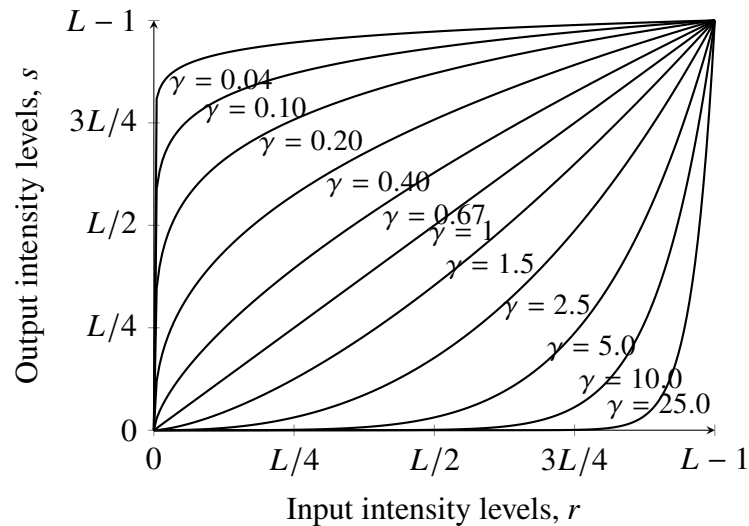


Figure 3.6: (c) Plot dari persamaan transformasi gamma $s = cr^\gamma$ untuk berbagai nilai γ (dengan $c = 1$ pada semua kasus). Setiap kurva diskalakan secara independen agar semuanya dapat ditampilkan dalam grafik yang sama. Fokus utama adalah pada bentuk kurva, bukan pada nilai relatifnya.

Perbandingan berbagai nilai γ dapat dilihat pada [FIGURE 3.6].

Mengapa disebut Gamma Correction? Monitor CRT secara fisik menampilkan kecerahan secara non-linier ($\gamma \approx 2.2$). Untuk mengkompensasinya, sinyal video dikodekan dengan $\gamma \approx 1/2.2 \approx 0.45$. Itulah asal usul istilah “gamma correction”.

2.3 Contoh Kode: Transformasi Intensitas dengan Python

Listing 1: Implementasi transformasi intensitas dasar

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from PIL import Image
4
5 # Load citra grayscale
6 img = np.array(Image.open("image.jpg").convert("L"), dtype=np.float64
7 )
8 L = 256 # level intensitas
9
10 # 1. Citra Negatif
11 img_negative = (L - 1) - img
12
13 # 2. Transformasi Log
14 c = 255 / np.log(1 + np.max(img))
15 img_log = c * np.log(1 + img)
16
17 # 3. Transformasi Gamma (Power-Law)
18 gamma = 0.4 # coba juga 1.0, 2.5

```

```

18 c = 1.0
19 img_gamma = c * np.power(img / 255.0, gamma) * 255.0
20
21 # Visualisasi
22 fig, axes = plt.subplots(1, 4, figsize=(16, 4))
23 titles = ['Original', 'Negative', 'Log', f'Gamma ({gamma})']
24 images = [img, img_negative, img_log, img_gamma]
25 for ax, title, im in zip(axes, titles, images):
26     ax.imshow(np.clip(im, 0, 255).astype(np.uint8), cmap='gray')
27     ax.set_title(title)
28     ax.axis('off')
29 plt.tight_layout()
30 plt.show()

```

3 Pemrosesan Histogram (Bagian 3.3)

3.1 Apa Itu Histogram Citra?

Histogram adalah “kartu laporan” distribusi kecerahan dalam sebuah citra. Secara formal, untuk citra digital dengan tingkat keabuan $[0, L - 1]$, histogram yang dinormalisasi (yang berfungsi sebagai estimasi probabilitas) didefinisikan sebagai:

$$p(r_k) = \frac{n_k}{MN} \quad (6)$$

di mana n_k adalah jumlah piksel dengan intensitas r_k , dan MN adalah total jumlah piksel dalam citra.

- Histogram condong ke **kiri**: citra terlalu **gelap**.
- Histogram condong ke **kanan**: citra terlalu **terang**.
- Histogram **sempit**: kontras **rendah**.
- Histogram **tersebar merata**: kontras **baik**.

3.2 Histogram Equalization (HE)

Histogram Equalization adalah teknik otomatis untuk meningkatkan kontras dengan cara menyebarkan distribusi intensitas yang menumpuk menjadi seragam (*uniform*).

Mekanisme Matematis:

Fungsi transformasi yang digunakan adalah *Cumulative Distribution Function* (CDF) dari histogram:

$$s_k = T(r_k) = (L - 1) \sum_{j=0}^k p_r(r_j) \quad (7)$$

Syarat agar transformasi valid: $T(r)$ harus meningkat secara monoton agar urutan terang-gelap tidak tertukar. CDF secara alami memenuhi syarat ini.

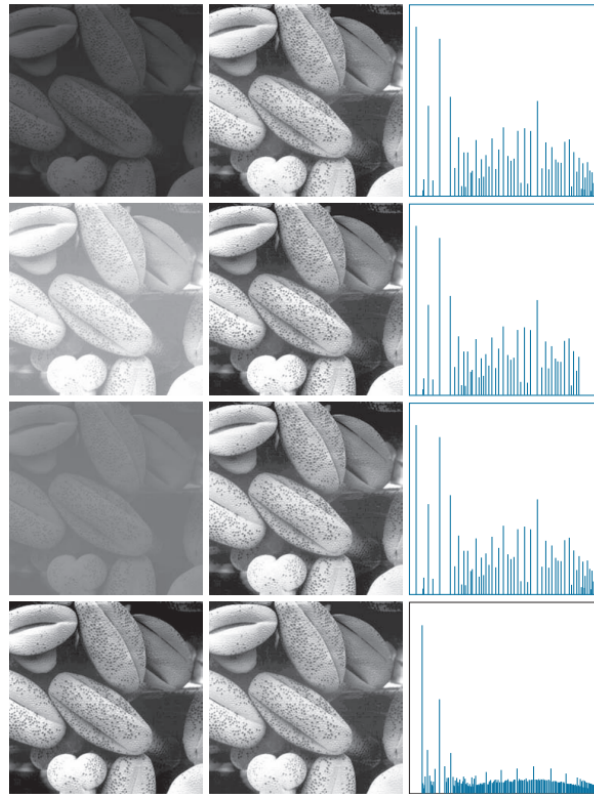


Figure 3.20: Kolom kiri: Citra dari Gambar 3.16. Kolom tengah: Citra hasil ekualisasi histogram yang bersesuaian. Kolom kanan: Histogram dari citra pada kolom tengah (bandingkan dengan histogram pada Gambar 3.16).

Hasil: Citra yang tadinya “buram” karena kontras rendah akan menjadi lebih tajam secara otomatis. Lihat [FIGURE 3.21].

Pertanyaan Reflektif: Jika HE bisa memperbaiki kontras secara otomatis, mengapa kita masih membutuhkan *Histogram Matching (Spesifikasi)*? Petunjuk: terkadang HE membuat citra terlihat “terlalu terang” atau tidak natural. Lihat [FIGURE 3.24].

3.3 Histogram Matching (Spesifikasi Histogram)

Histogram Matching memungkinkan kita menentukan *bentuk histogram target* yang diinginkan, bukan hanya seragam. Ini berguna ketika kita ingin mencocokkan penampilan dua citra berbeda (misalnya dalam mosaik satelit).

Langkah-langkahnya:

1. Hitung CDF dari citra asal: $s = T(r)$.
2. Hitung CDF dari distribusi target: $G(z)$.
3. Cari pemetaan $z = G^{-1}(s)$.

3.4 Contoh Kode: Histogram Equalization

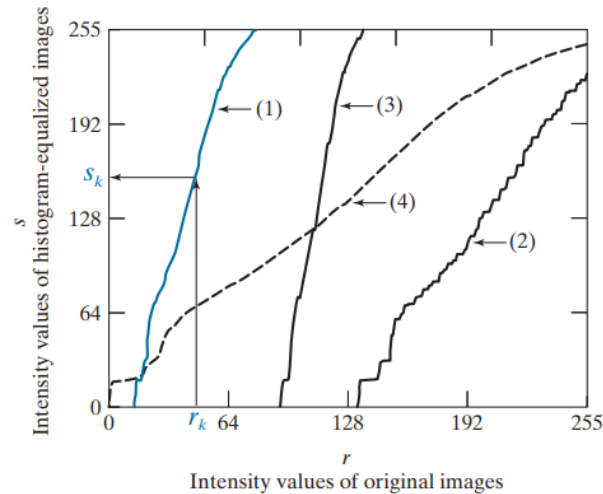


Figure 3.21: Fungsi transformasi untuk ekualisasi histogram. Transformasi (1) hingga (4) diperoleh menggunakan Persamaan (3-15) dan histogram dari citra pada kolom kiri Gambar 3.20. Pemetaan satu nilai intensitas r_k pada citra 1 ke nilai yang bersesuaian s_k ditunjukkan.

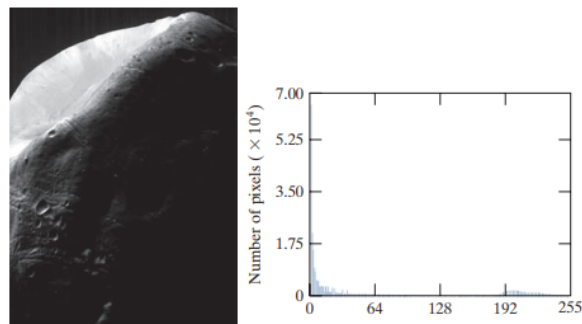


Figure 3.23: (a) Sebuah citra, dan (b) histogramnya..

Listing 2: Histogram Equalization dari nol (tanpa OpenCV)

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4
5 def histogram_equalization(img_gray):
6     """
7     img_gray: numpy array 2D, nilai [0, 255]
8     """
9     L = 256
10    M, N = img_gray.shape
11    total_pixels = M * N
12
13    # 1. Hitung histogram
14    hist = np.zeros(L, dtype=np.float64)
15    for val in img_gray.flatten():
16        hist[val] += 1
17
18    # 2. Normalisasi -> PDF

```

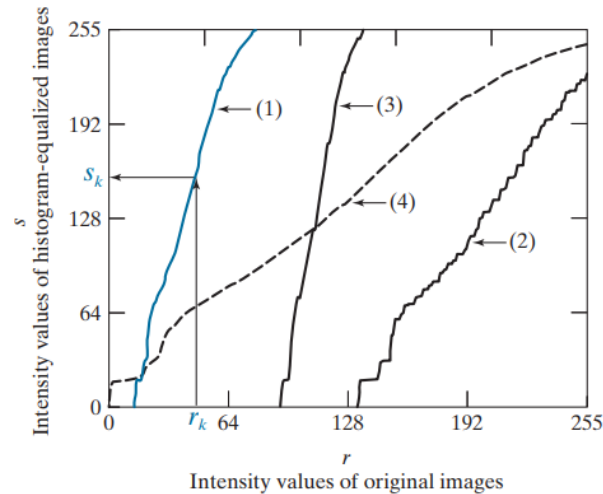



Figure 3.24: (a) Transformasi ekualisasi histogram yang diperoleh menggunakan histogram pada Gambar 3.23(b). (b) Citra hasil ekualisasi histogram. (c) Histogram dari citra yang telah diekualisasi.

```

19 pdf = hist / total_pixels
20
21 # 3. Hitung CDF
22 cdf = np.cumsum(pdf)
23
24 # 4. Buat lookup table (LUT) transformasi
25 lut = np.floor((L - 1) * cdf).astype(np.uint8)
26
27 # 5. Terapkan ke citra
28 img_eq = lut[img_gray]
29 return img_eq, hist, lut
30
31 img = np.array(Image.open("image.jpg").convert("L"))
32 img_eq, hist, lut = histogram_equalization(img)
33
34 fig, axes = plt.subplots(2, 2, figsize=(10, 8))
35 axes[0, 0].imshow(img, cmap='gray'); axes[0, 0].set_title('Citra Asli')
36 axes[0, 1].imshow(img_eq, cmap='gray'); axes[0, 1].set_title('Setelah HE')
37 axes[1, 0].bar(range(256), hist, width=1); axes[1, 0].set_title('Histogram Asli')
38 axes[1, 1].hist(img_eq.flatten(), bins=256); axes[1, 1].set_title('Histogram Setelah HE')
39 plt.tight_layout()
40 plt.show()

```

Tips Praktis: Dengan OpenCV, HE bisa dilakukan hanya dengan satu baris: `cv2.equalizeHist(img)`. Implementasi manual di atas berguna untuk memahami mekanismenya secara mendalam.

4 Dasar Penyaringan Spasial (Bagian 3.4)

4.1 Mekanisme: Neighborhood Processing

Berbeda dengan *point processing*, penyaringan spasial menggunakan **Neighborhood Processing**. Sebuah jendela kecil bernama **kernel** (juga disebut *mask*, *filter*, atau *template*) digeser di atas citra piksel demi piksel.

Di setiap posisi, hasil perkalian elemen-per-elemen antara kernel dan area citra di bawahnya dijumlahkan untuk menghasilkan nilai piksel baru.

4.2 Korelasi vs. Konvolusi

Dua operasi ini sering tertukar. Perbedaan teknisnya penting:

Operasi	Definisi	Penggunaan
Korelasi	Kalikan kernel langsung dengan area citra	Filter praktis, CNN
Konvolusi	Kernel diputar 180° lalu dikalikan	Teori sistem linear

Catatan Penting: Untuk kernel yang *simetris* (seperti Gaussian), korelasi dan konvolusi menghasilkan output yang *identik*. Perbedaan hanya muncul pada kernel asimetris.

Rumus Konvolusi 2D:

$$(w * f)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x - s, y - t) \quad (8)$$

Rumus Korelasi 2D:

$$(w \star f)(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t) \quad (9)$$

4.3 Padding: Menangani Tepi Citra

Saat kernel diaplikasikan di tepi citra, sebagian area kernel berada di luar batas citra. Beberapa strategi penanganan:

- **Zero Padding:** isi area luar dengan nol. Paling sederhana, bisa membuat tepi gelap.
- **Reflect Padding:** cerminkan nilai piksel. Lebih natural untuk banyak kasus.
- **Replicate Padding:** ulangi piksel tepi. Umum digunakan.

4.4 Contoh Kode: Konvolusi 2D Manual

Listing 3: Implementasi konvolusi 2D dari nol

```

1 import numpy as np
2
3 def convolve2d(image, kernel):
4     """
5     Konvolusi 2D dengan zero-padding.
6     image : numpy array 2D
7     kernel: numpy array 2D (ukuran ganjil x ganjil)
8     """
9     # Rotasi kernel 180 derajat untuk konvolusi
10    kernel_rot = np.rot90(kernel, 2)
11
12    kh, kw = kernel_rot.shape
13    pad_h, pad_w = kh // 2, kw // 2
14
15    # Zero padding
16    padded = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='
    constant')
17
18    H, W = image.shape
19    output = np.zeros_like(image, dtype=np.float64)
20
21    for i in range(H):
22        for j in range(W):
23            region = padded[i:i+kh, j:j+kw]
24            output[i, j] = np.sum(region * kernel_rot)
25
26    return output
27
28 # Contoh penggunaan dengan kernel sederhana (identitas)
29 identity_kernel = np.array([[0, 0, 0],
30                             [0, 1, 0],
31                             [0, 0, 0]])
32 img = np.array([[100, 120, 80],
33                 [90, 150, 110],
34                 [70, 130, 95]])
35 result = convolve2d(img, identity_kernel)
36 print(result) # Harus sama dengan img

```

5 Penyaringan Smoothing / Lowpass (Bagian 3.5)

5.1 Tujuan

Filter *smoothing* bertujuan mengurangi noise atau menghilangkan detail kecil yang tidak relevan dengan cara mengaburkan (*blurring*) citra. Dalam perspektif frekuensi, filter ini menekan komponen frekuensi tinggi (detail) sambil mempertahankan komponen frekuensi rendah (struktur besar).

5.2 Box Filter (Average Filter)

Mengambil rata-rata sederhana dari semua piksel dalam jendela berukuran $n \times n$.

$$\mathbf{K}_{\text{box}} = \frac{1}{n^2} \begin{bmatrix} 1 & 1 & \cdots & 1 \\ 1 & 1 & \cdots & 1 \\ \vdots & & \ddots & \vdots \\ 1 & 1 & \cdots & 1 \end{bmatrix} \quad (10)$$

Kelebihan: Sederhana dan cepat secara komputasi.

Kekurangan: Menghasilkan efek “kotak” (*ringing*) pada tepi karena semua tetangga diberi bobot sama.

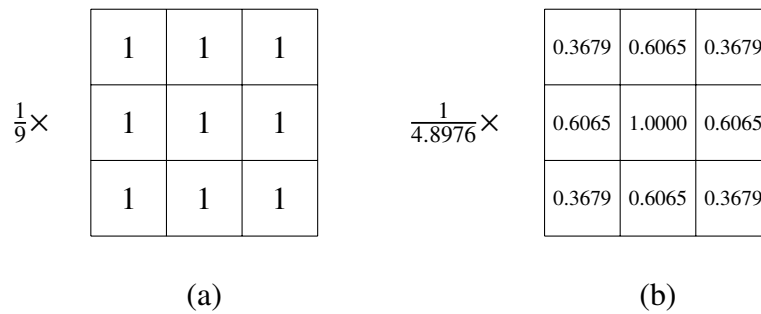


Figure 3.31: Contoh kernel smoothing: (a) adalah kernel kotak (*box kernel*); (b) adalah kernel Gaussian.

Lihat hasil box filter pada [FIGURE 3.31a].

5.3 Gaussian Filter

Memberikan bobot lebih besar pada piksel yang lebih dekat ke pusat kernel. Dimodelkan dengan fungsi Gaussian 2D:

$$G(s, t) = K \cdot e^{-\frac{s^2 + t^2}{2\sigma^2}} \quad (11)$$

di mana σ adalah standar deviasi yang mengontrol “lebar” pengaburan, dan K adalah konstanta normalisasi.

Kelebihan:

- Hasil lebih halus dan natural.
- Bersifat *isotropik* (efek sama ke segala arah).
- Dapat dipisahkan menjadi dua filter 1D (efisien secara komputasi).

Contoh kernel Gaussian 3×3 ($\sigma \approx 1$):

$$\mathbf{K}_{\text{gauss}} \approx \frac{1}{16} \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix} \quad (12)$$

Lihat hasil Gaussian filter pada [FIGURE 3.31b].

5.4 Median Filter (Non-Linear)

Berbeda dari dua filter sebelumnya, median filter *bukan* filter linear. Nilai piksel baru adalah **nilai median** dari semua piksel dalam jendela.

Keunggulan utama: Sangat efektif menghilangkan *salt-and-pepper noise* (piksel yang tiba-tiba bernilai 0 atau 255) tanpa merusak tepi (*edge*) citra – sesuatu yang tidak bisa dilakukan filter linear dengan baik.

Mengapa Median Filter Bekerja? Piksel *salt-and-pepper* adalah *outlier* yang nilainya jauh dari tetangga sekitarnya. Operasi rata-rata (mean) dipengaruhi oleh outlier, tapi median tidak – median selalu memilih nilai “tengah” yang lebih representatif.

5.5 Contoh Kode: Perbandingan Filter Smoothing

Listing 4: Membandingkan Box, Gaussian, dan Median Filter

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4 from scipy import ndimage
5
6 # Load citra dan tambah salt-and-pepper noise
7 img = np.array(Image.open("image.jpg").convert("L"), dtype=np.float64
8 )
9 noise_mask = np.random.choice([0, 1, 2], size=img.shape, p=[0.05,
10 0.05, 0.90])
11 img_noisy = img.copy()
12 img_noisy[noise_mask == 0] = 0      # pepper
13 img_noisy[noise_mask == 1] = 255   # salt
14
15 # 1. Box Filter
16 box_kernel = np.ones((5, 5)) / 25.0
17 img_box = ndimage.convolve(img_noisy, box_kernel)
18
19 # 2. Gaussian Filter
20 img_gauss = ndimage.gaussian_filter(img_noisy, sigma=1.5)
21
22 # 3. Median Filter
23 img_median = ndimage.median_filter(img_noisy, size=5)
24
25 # Visualisasi
26 fig, axes = plt.subplots(1, 4, figsize=(16, 4))
27 for ax, title, im in zip(axes,
28 ['Noisy', 'Box (5x5)', 'Gaussian (sigma=1.5)', 'Median (5x5)'],
29 [img_noisy, img_box, img_gauss, img_median]):
30     ax.imshow(np.clip(im, 0, 255).astype(np.uint8), cmap='gray')
31     ax.set_title(title); ax.axis('off')
32 plt.tight_layout(); plt.show()

```

6 Penyaringan Sharpening / Highpass (Bagian 3.6)

6.1 Prinsip Dasar: Penajaman sebagai Diferensiasi

Jika *smoothing* identik dengan integrasi (menghaluskan perubahan), maka **sharpening** identik dengan **diferensiasi** (memperkuat perubahan). Tepi pada citra adalah area di mana intensitas berubah tajam – persis apa yang dideteksi oleh turunan.

6.2 Laplacian: Turunan Orde Kedua

Laplacian adalah operator diferensial orde kedua yang paling populer untuk penajaman citra karena bersifat *isotropik*.

Definisi kontinu:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} \quad (13)$$

Bentuk diskrit (aproksimasi untuk piksel):

$$\nabla^2 f(x, y) = f(x+1, y) + f(x-1, y) + f(x, y+1) + f(x, y-1) - 4f(x, y) \quad (14)$$

Kernel Laplacian yang bersesuaian:

$$\mathbf{L}_1 = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{atau (dengan diagonal):} \quad \mathbf{L}_2 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad (15)$$

Cara menggunakannya untuk menajamkan citra:

$$g(x, y) = f(x, y) - \nabla^2 f(x, y) \quad (16)$$

(tanda minus karena Laplacian menghasilkan nilai negatif di puncak tepi)

Lihat perbandingan sebelum dan sesudah penajaman dengan Laplacian pada **[FIGURE 3.46]**.

6.3 Gradient: Turunan Orde Pertama

Untuk mendeteksi *arah* tepi, digunakan gradient. Operator populer:

Sobel Operator:

$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \quad \mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \quad (17)$$

Magnitudo gradient:

$$M = \sqrt{G_x^2 + G_y^2} \quad (18)$$

Perbandingan turunan orde pertama vs. orde kedua dalam mendeteksi tepi dan detail halus dapat dilihat pada **[FIGURE 3.44]**.

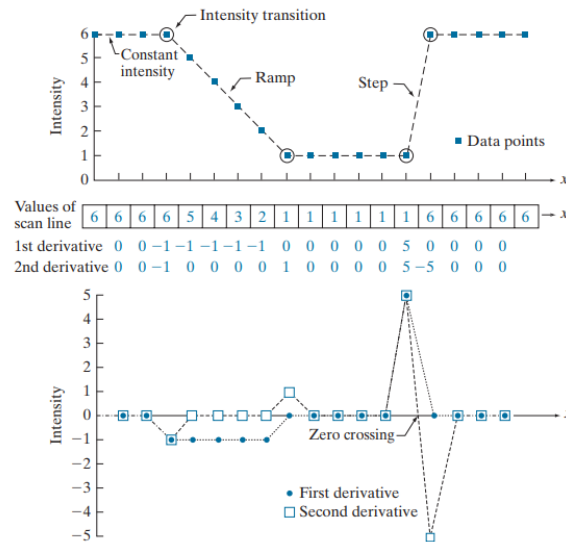


Figure 3.44: (a) Sebagian dari garis pindai horizontal pada sebuah citra, yang menunjukkan tepi ramp dan step, serta segmen konstan. (b) Nilai-nilai garis pindai tersebut dan turunannya. (c) Plot turunan, yang menunjukkan titik potong nol. Pada (a) dan (c), titik-titik dihubungkan dengan garis putus-putus sebagai bantuan visual..

0	1	0
1	-4	1
0	1	0

(a)

1	1	1
1	-8	1
1	1	1

(b)

0	-1	0
-1	4	-1
0	-1	0

(c)

-1	-1	-1
-1	8	-1
-1	-1	-1

(d)

Figure 3.45: (a) Kernel Laplacian yang digunakan untuk mengimplementasikan Persamaan (3-53). (b) Kernel yang digunakan untuk mengimplementasikan perluasan dari persamaan tersebut yang mencakup elemen diagonal. (c) dan (d) Dua kernel Laplacian lainnya.

6.4 Unsharp Masking

Unsharp Masking adalah teknik penajaman klasik yang berasal dari dunia fotografi film. Langkah-langkahnya:

1. Buat citra blur: $\bar{f}(x, y) = \text{Gaussian}(f(x, y))$
2. Buat “masker”: $g_{\text{mask}}(x, y) = f(x, y) - \bar{f}(x, y)$
3. Tambahkan masker ke citra asli:

$$g(x, y) = f(x, y) + k \cdot g_{\text{mask}}(x, y) \quad (19)$$

di mana k adalah faktor penguat (*boost factor*).

Ketika $k > 1$, teknik ini disebut **Highboost Filtering**.

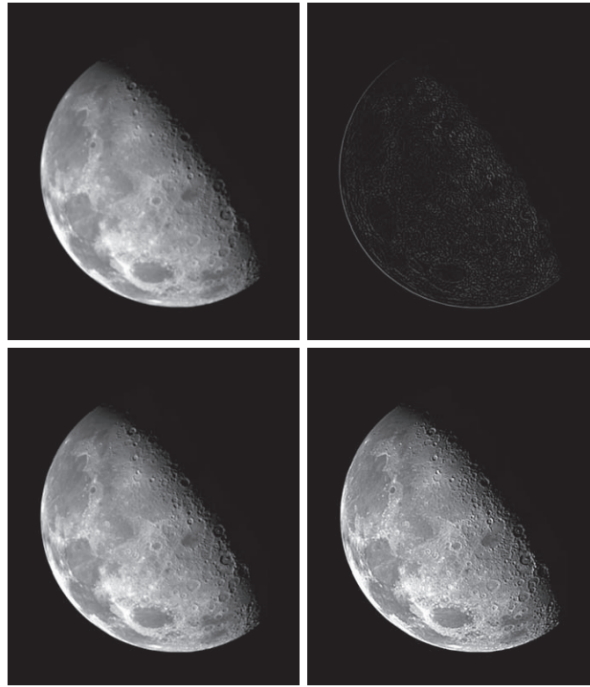


Figure 3.46: (a) Citra buram dari Kutub Utara bulan. (b) Citra Laplasian yang diperoleh menggunakan kernel pada Gambar 3.45(a). (c) Citra yang dipertajam menggunakan Persamaan (3-54) dengan $c=-1$ $c=-1$ $c=-1$. (d) Citra yang dipertajam menggunakan prosedur yang sama, tetapi dengan kernel pada Gambar 3.45(b). (Citra asli merupakan milik NASA.)

6.5 Contoh Kode: Sharpening dengan Laplacian dan Unsharp Masking

Listing 5: Penajaman citra: Laplacian dan Unsharp Masking

```

1 import numpy as np
2 from PIL import Image
3 import matplotlib.pyplot as plt
4 from scipy import ndimage
5
6 img = np.array(Image.open("image.jpg").convert("L"), dtype=np.float64
7 )
8 # --- 1. Laplacian Sharpening ---
9 laplacian_kernel = np.array([[ 0,  1,  0],
10                             [ 1, -4,  1],
11                             [ 0,  1,  0]])
12
13 laplacian = ndimage.convolve(img, laplacian_kernel)
14
15 # Kurangi citra asli dengan Laplacian (tanda minus karena definisi
16   negatif)
17 img_laplacian = img - laplacian
18 img_laplacian = np.clip(img_laplacian, 0, 255)
19
20 # --- 2. Unsharp Masking ---
21 sigma = 3.0

```



```

21 k = 1.5 # boost factor (k > 1 = highboost filtering)
22
23 img_blur = ndimage.gaussian_filter(img, sigma=sigma)
24 mask = img - img_blur
25 img_unsharp = img + k * mask
26 img_unsharp = np.clip(img_unsharp, 0, 255)
27
28 # Visualisasi
29 fig, axes = plt.subplots(1, 4, figsize=(16, 4))
30 data = [
31     ('Original', img),
32     ('Laplacian Response', laplacian + 128), # shift untuk
        visualisasi
33     ('Laplacian Sharpened', img_laplacian),
34     (f'Unsharp Mask (k={k})', img_unsharp),
35 ]
36 for ax, (title, im) in zip(axes, data):
37     ax.imshow(np.clip(im, 0, 255).astype(np.uint8), cmap='gray')
38     ax.set_title(title); ax.axis('off')
39 plt.tight_layout(); plt.show()

```

7 Ringkasan Perbandingan Filter

Filter	Tipe	Tujuan	Noise	Tepi
Box	Linear	Blur/Smoothing	Kurang efektif	Rusak
Gaussian	Linear	Blur/Smoothing	Efektif	Agak rusak
Median	Non-linear	Denoising	Sangat efektif	Terjaga
Laplacian	Linear	Sharpening	Memperkuat noise	Diperkuat
Unsharp Mask	Linear	Sharpening	Memperkuat noise	Diperkuat
Sobel	Linear	Deteksi tepi	Sensitif	Arah tepi

8 Mini-Latihan (Active Learning)

Latihan 1. Kernel Laplacian pada Area Seragam

Dalam Python (menggunakan numpy), jika Anda memiliki kernel Laplacian:

```

1 kernel = np.array([[0, 1, 0],
2                    [1, -4, 1],
3                    [0, 1, 0]])

```

Apa yang terjadi jika kernel ini diaplikasikan pada area citra yang intensitasnya seragam (misalnya semua piksel bernilai 100)? Mengapa hasilnya nol?

Petunjuk: Hitung secara manual: $0 + 100 + 0 + 100 + (-4 \times 100) + 100 + 0 + 100 + 0 = ?$

Apa yang dapat Anda simpulkan tentang sifat Laplacian terhadap wilayah datar?

Latihan 2. Orde Pertama vs. Orde Kedua

Berdasarkan [FIGURE 3.44], jelaskan mengapa turunan orde kedua (Laplacian) lebih baik dalam mendeteksi detail halus dibandingkan turunan orde pertama (Gradient/Sobel).

Petunjuk: Perhatikan respons masing-masing turunan terhadap: (a) tepi tajam (*step edge*), (b) garis tipis, dan (c) area transisi gradual. Turunan orde berapa yang menghasilkan respons puncak (*zero crossing*)?

Latihan 3. Eksperimen Gamma Correction

Dengan menggunakan kode di Bagian 2.3, coba nilai $\gamma \in \{0.1, 0.4, 1.0, 2.0, 5.0\}$ pada sebuah citra foto wajah. Dokumentasikan perubahan yang terjadi pada: (a) kecerahan keseluruhan, (b) detail pada area bayangan, dan (c) detail pada area terang.

Latihan 4. Membandingkan HE vs. Tanpa HE

Ambil sebuah citra yang sangat gelap (misalnya foto malam hari). Terapkan Histogram Equalization, kemudian bandingkan hasilnya dengan menerapkan transformasi gamma $\gamma = 0.3$ secara manual. Mana yang lebih natural? Diskusikan kelebihan dan kekurangan masing-masing pendekatan.